

WEEK 12

Task 1

Skill objective:

To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

Implementation / work:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void sig_handler(int signo) {    if (signo == SIGUSR1) {        printf("[Signal Handler] Caught
SIGUSR1 (Signal %d). Shell stability intact.\n", signo);    } }

int main() {    struct
sigaction sa;    sa.sa_handler
= sig_handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = SA_RESTART;

    if (sigaction(SIGUSR1, &sa, NULL) < 0) {
perror("sigaction");        return 1;    }

    printf("[Task 1] Registered robust sigaction handler with SA_RESTART.\n");
printf("Raising SIGUSR1 test event internally...\n");    raise(SIGUSR1);

    printf("Execution resumed cleanly past signal event without crash.\n");
return 0; }
```

Output:

```
naveen123@LAPTOP-4NVQJ2C7:~$ nano exp12s.c
naveen123@LAPTOP-4NVQJ2C7:~$ gcc exp12s.c -o exp12s
naveen123@LAPTOP-4NVQJ2C7:~$ ./exp12s
[Task 1] Registered robust sigaction handler with SA_RESTART.
Raising SIGUSR1 test event internally...
[Signal Handler] Caught SIGUSR1 (Signal 10). Shell stability intact.
Execution resumed cleanly past signal event without crash.
```

Task 2

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

Implementation / work:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
```

Skill objective:

```
pid_t fg_pid = 0;

void sigint_handler(int sig) {    if (fg_pid > 0) {          printf("\n[SIGINT Handler] Forwarding
Ctrl+C (SIGINT) to foreground PID %d...\n", fg_pid);          kill(fg_pid, SIGINT);      } else {
printf("\n[SIGINT Handler] Shell root protected. Refreshing prompt: teja27@TEJA:~$ \n");      } }

int main() {    struct sigaction
sa;    sa.sa_handler =
sigint_handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = SA_RESTART;
sigaction(SIGINT, &sa, NULL);

    printf("[Task 2] Testing SIGINT protection on interactive shell:\n");
printf("1. Triggering SIGINT without foreground job:\n");    raise(SIGINT);

    printf("2. Triggering SIGINT with foreground job active:\n");
fg_pid = 4152;    raise(SIGINT);

    printf("Shell process survived all interrupt attempts safely.\n");
return 0; } Output:
```

Skill objective:

```
naveen123@LAPTOP-4NVQJ2C7:~$ nano exp12s2.c
naveen123@LAPTOP-4NVQJ2C7:~$ gcc exp12s2.c -o exp12s2
naveen123@LAPTOP-4NVQJ2C7:~$ ./exp12s2
[Task 2] Testing SIGINT protection on interactive shell:
1. Triggering SIGINT without foreground job:

[SIGINT Handler] Shell root protected. Refreshing prompt: naveen123@LAPTOP-4NVQJ2C7:~$
2. Triggering SIGINT with foreground job active:

[SIGINT Handler] Forwarding Ctrl+C (SIGINT) to foreground PID 4152...
Shell process survived all interrupt attempts safely.
naveen123@LAPTOP-4NVQJ2C7:~$
```

Task 3

To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

Implementation / work:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void sigtstp_handler(int sig) {
    printf("\n[SIGTSTP Handler] Ctrl+Z received!
Suspending foreground process...\n");
    printf("[1]+ Stopped
my_compute_job\n");
    printf("Preserving state into Job Table with status:
STOPPED\n");
    printf("Prompt returned: teja27@TEJA:~$ \n");
}

int main() {
    struct sigaction
    sa;
    sa.sa_handler =
    sigtstp_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGTSTP, &sa, NULL);

    printf("[Task 3] Testing Ctrl+Z (SIGTSTP) process suspension workflow:\n");
    raise(SIGTSTP);
    printf("Process suspension workflow validated
successfully.\n");
    return 0; }
```

Output:

Skill objective:

```

Shell process survived all interrupt attempts safely.
naveen123@LAPTOP-4NVQJ2C7:~$ nano exp12s3.c
naveen123@LAPTOP-4NVQJ2C7:~$ gcc exp12s3.c -o exp12s3
naveen123@LAPTOP-4NVQJ2C7:~$ ./exp12s3
[Task 3] Testing Ctrl+Z (SIGTSTP) process suspension workflow:

[SIGTSTP Handler] Ctrl+Z received! Suspending foreground process...
[1]+ Stopped my_compute_job
Preserving state into Job Table with status: STOPPED
Prompt returned: naveen123@LAPTOP-4NVQJ2C7:~$
Process suspension workflow validated successfully.
naveen123@LAPTOP-4NVQJ2C7:~$

```

Task 4

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

Implementation / work:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        /* Child creates its
        own process group matching its PID */
        setpgid(0, 0);
        printf("[Child
        PGID] Isolated Child PID: %d, PGID: %d\n", getpid(), getpgrp());
        _exit(0);
    }

    setpgid(pid, pid); /* Avoid race between parent and child */
    printf("[Parent
    Shell] Shell PID: %d, PGID: %d\n", getpid(), getpgrp());
    printf("[Parent Shell]
    Assigned child PID %d to Process Group %d\n", pid, pid);

    waitpid(pid, NULL, 0);
    printf("[Parent Shell] Child exited. Restored terminal control to
    shell PGID %d.\n", getpgrp());
    return 0; }

```

Output:

Skill objective:

```
naveen123@LAPTOP-4NVQJ2C7:~$ nano exp12s4.c
naveen123@LAPTOP-4NVQJ2C7:~$ gcc exp12s4.c -o exp12s4
naveen123@LAPTOP-4NVQJ2C7:~$ ./exp12s4
[Parent Shell] Shell PID: 1901, PGID: 1901
[Parent Shell] Assigned child PID 1902 to Process Group 1902
[Child PGID] Isolated Child PID: 1902, PGID: 1902
[Parent Shell] Child exited. Restored terminal control to shell PGID 1901.
naveen123@LAPTOP-4NVQJ2C7:~$ |
```